Лабораторная работа №4

Продвинутое использование git

Лебедев С. А.

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**
4.1 Установка программного обеспечения 8
4.1.1 Установка git-flow 8
4.1.2 Установка Node.js и pnpm 8
4.1.3 Настройка pnpm 9
4.2 Создание репозитория git 10
4.2.1 Инициализация локального репозитория 10
4.2.2 Подключение удалённого репозитория 10
4.3 Конфигурация общепринятых коммитов 11
4.3.1 Инициализация проекта Node.js 11
4.3.2 Настройка commitizen 11
4.3.3 Создание коммита с использованием commitizen 12
4.4 Конфигурация git-flow 13
4.4.1 Инициализация git-flow 13
4.4.2 Создание первого релиза 13
4.5 Работа с репозиторием git 14
4.5.1 Завершение работы с функциональной веткой 14
4.5.2 Отправка изменений в удалённый репозиторий 15
4.5.3 Создание релиза на GitHub 15

5 Контрольные вопросы **17**

6 Выводы **18**

Список литературы **19**

Список иллюстраций

4.1 Установка gitflow через репозиторий Copr 8
4.2 Установка Node.js и pnpm 9
4.3 Настройка pnpm 9
4.4 Создание тестового репозитория и первый коммит 10
4.5 Подключение удалённого репозитория и первый push 11
4.6 Инициализация проекта Node.js 11
4.7 Редактирование package.json для настройки commitizen 12
4.8 Создание коммита с использованием commitizen 12
4.9 Инициализация git-flow 13
4.10 Создание релиза 1.0.0 и генерация changelog 14
4.11 Завершение feature-ветки и создание нового релиза 15
4.12 Отправка всех веток и тегов на GitHub 15
4.13 Создание релиза v1.2.3 на GitHub 16

Список таблиц

1 Цель работы

Целью данной работы является получение навыков правильной работы с репозиториями git.

2 Задание

1. Выполнить работу для тестового репозитория.
2. Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 Теоретическое введение

Gitflow Workflow — модель ветвления для git, опубликованная и популяризованная Винсентом Дриссеном. Предполагает выстраивание строгой модели ветвления с учётом выпуска проекта и отлично подходит для организации рабочего процесса на основе релизов.

Семантическое версионирование задаёт версию в виде кортежа `МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ`, где каждый компонент увеличивается при соответствующем типе изменений: обратно несовместимых, новой функциональности и исправлений соответственно.

Conventional Commits — спецификация соглашений о написании сообщений коммитов. Совместима с SemVer и регламентирует структуру и основные типы коммитов (`feat:`, `fix:`, `docs:`, `chore:` и др.).

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

4.1.1 Установка git-flow

Для установки инструмента git-flow подключён дополнительный репозиторий Copr и выполнена установка пакета `gitflow` (рис. 4.1).

```
sudo dnf copr enable elegos/gitflow
sudo dnf install gitflow
```

Рис. 4.1: Установка gitflow через репозиторий Copr

4.1.2 Установка Node.js и pnpm

Установлены среда Node.js и менеджер пакетов pnpm, необходимые для работы инструментов conventional commits и генерации changelog (рис. 4.2).

```
sudo dnf install nodejs pnpm
```

Рис. 4.2: Установка Node.js и pnpm

4.1.3 Настройка pnpm

Выполнена настройка pnpm: добавлен путь к исполняемым файлам в переменную среды PATH (рис. 4.3).

```
pnpm setup
source ~/.bashrc
```

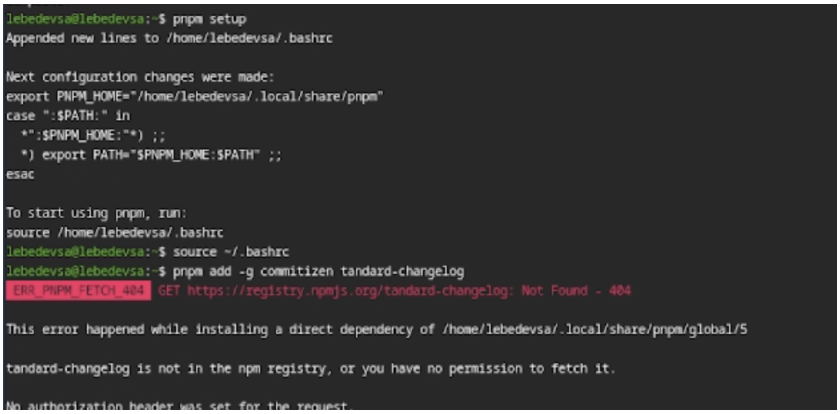

Рис. 4.3: Настройка pnpm

4.2 Создание репозитория git

4.2.1 Инициализация локального репозитория

Создан тестовый Git-репозиторий и выполнен первый коммит (рис. 4.4).

```
mkdir git-extended
cd git-extended
touch README.md
git init
git add .
git commit -m "first commit"
```

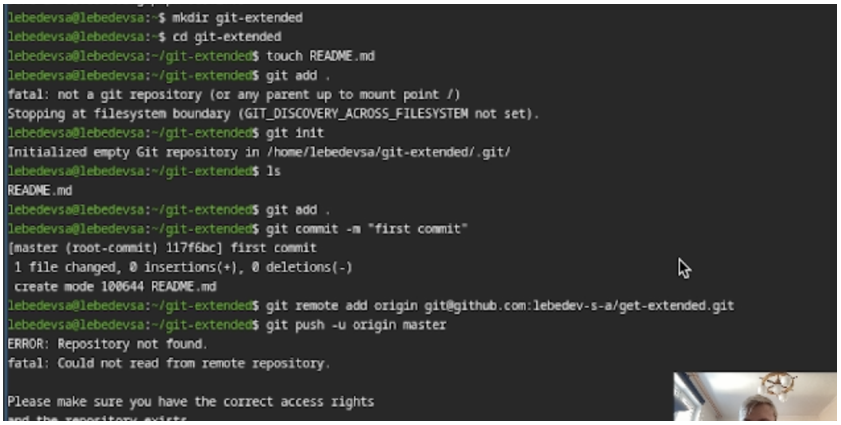

Рис. 4.4: Создание тестового репозитория и первый коммит

4.2.2 Подключение удалённого репозитория

Репозиторий подключён к GitHub и выполнена первичная отправка изменений (рис. 4.5).

```
git remote add origin git@github.com:lebedev-s-a/git-extended.git
git push -u origin master
```

```
lebedevsa@lebedevsa:~/git-extended$ git remote add origin git@github.com:lebedev-s-a/get-extended.git
lebedevsa@lebedevsa:~/git-extended$ git push -u origin master
ERROR: Repository not found.
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
lebedevsa@lebedevsa:~/git-extended$ git remote remove origin
lebedevsa@lebedevsa:~/git-extended$ git remote add origin git@github.com:lebedev-s-a/git-extended.git
lebedevsa@lebedevsa:~/git-extended$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 868 bytes | 868.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:lebedev-s-a/git-extended.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Рис. 4.5: Подключение удалённого репозитория и первый push

4.3 Конфигурация общепринятых коммитов

4.3.1 Инициализация проекта Node.js

Выполнена инициализация проекта Node.js, создан файл `package.json` с информацией о проекте (рис. 4.6).

```
pnpm init
```

```
lebedevsa@lebedevsa:~/git-extended$ pnpm init
Wrote to /home/lebedevsa/git-extended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
```

Рис. 4.6: Инициализация проекта Node.js

4.3.2 Настройка commitizen

В файл `package.json` добавлена конфигурация для commitizen, позволяющая использовать стандартизированные сообщения коммитов (рис. 4.7).

```
"config": {
  "commitizen": {
    "path": "cz-conventional-changelog"
```

```
  }
}
```

Рис. 4.7: Редактирование package.json для настройки commitizen

4.3.3 Создание коммита с использованием commitizen

Выполнен коммит с помощью интерактивного инструмента `git cz`. Выбран тип коммита `docs` (рис. 4.8).

```
git cz
```

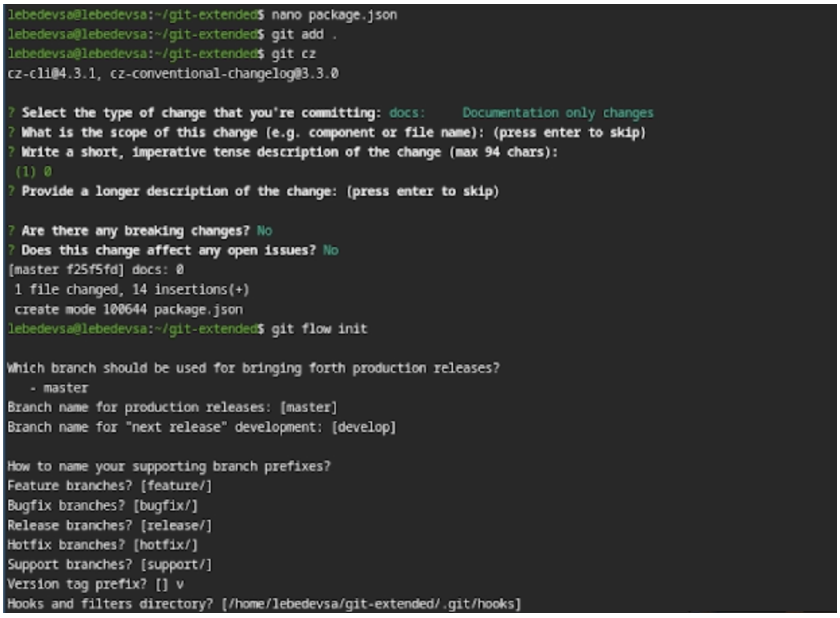

Рис. 4.8: Создание коммита с использованием commitizen

4.4 Конфигурация git-flow

4.4.1 Инициализация git-flow

Выполнена инициализация git-flow с настройкой основных веток (`master`, `develop`) и префиксов для `feature/`, `release/`, `hotfix/`. Префикс для ярлыков установлен в v (рис. 4.9).

```
git flow init
```

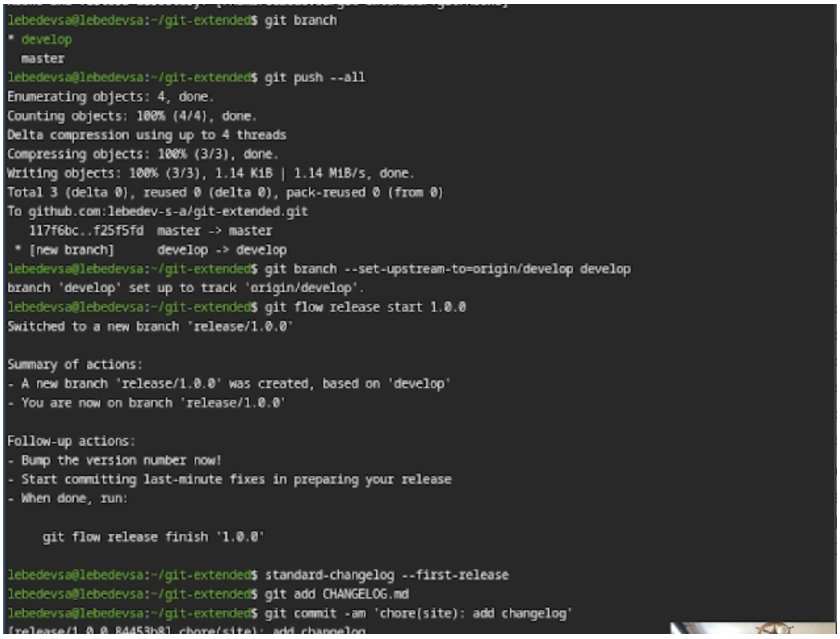

Рис. 4.9: Инициализация git-flow

4.4.2 Создание первого релиза

Создана ветка `release/1.0.0`, сгенерирован файл `CHANGELOG.md` и выполнен соответствующий коммит (рис. 4.10).

```
git flow release start 1.0.0
standard-changelog --first-release
git add CHANGELOG.md
git commit -am "chore(site): add changelog"
```

```
lebedevsa@lebedevsa:~/git-extended$ git push --all
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 2.68 KiB | 2.68 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:lebedev-s-a/git-extended.git
   f25f5fd..81ccccf  develop -> develop
   f25f5fd..0df3fe9  master -> master
lebedevsa@lebedevsa:~/git-extended$ git push --tags
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 160 bytes | 160.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:lebedev-s-a/git-extended.git
 * [new tag]         v1.0.0 -> v1.0.0
lebedevsa@lebedevsa:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/lebedev-s-a/git-extended/releases/tag/v1.0.0
```

Рис. 4.10: Создание релиза 1.0.0 и генерация changelog

4.5 Работа с репозиторием git

4.5.1 Завершение работы с функциональной веткой

Функциональная ветка `feature/feature_branch` слита с веткой `develop` и удалена. После этого создана новая релизная ветка `release/1.2.3` (рис. 4.11).

```
git flow feature finish feature_branch
git flow release start 1.2.3
```

```
lebedevsa@lebedevsa:~/git-extended$ git flow feature finish feature_branch
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
Updating 81ccccf..324459d
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
Deleted branch feature/feature_branch (was 324459d).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
- You are now on branch 'develop'

lebedevsa@lebedevsa:~/git-extended$ git flow release start 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Switched to a new branch 'release/1.2.3'

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'
```

Рис. 4.11: Завершение feature-ветки и создание нового релиза

4.5.2 Отправка изменений в удалённый репозиторий

Все ветки и теги отправлены в удалённый репозиторий на GitHub (рис. 4.12).

```
git push --all
git push --tags
```

```
lebedevsa@lebedevsa:~/git-extended$ git push --all
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 3.81 KiB | 3.81 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lebedev-s-a/git-extended.git
   81ccccf..d392482  develop -> develop
   0df3fe9..6cd4892  master -> master
lebedevsa@lebedevsa:~/git-extended$ git push --tags
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 160 bytes | 160.00 KiB/s, done.
```

Рис. 4.12: Отправка всех веток и тегов на GitHub

4.5.3 Создание релиза на GitHub

С помощью утилиты gh создан релиз `v1.2.3` на GitHub. Описание релиза сформировано из файла `CHANGELOG.md` (рис. 4.13).

```
gh release create v1.2.3 -F CHANGELOG.md
```

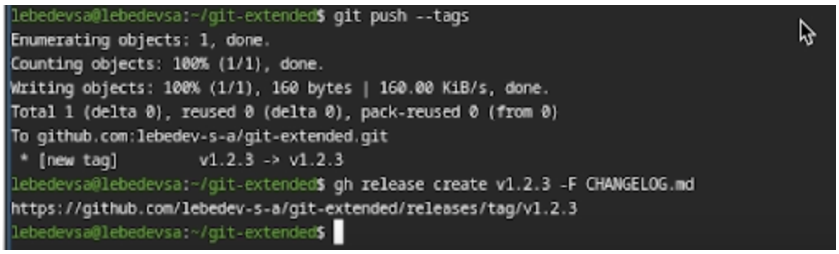

Рис. 4.13: Создание релиза v1.2.3 на GitHub

5 Контрольные вопросы

1. Что такое Gitflow Workflow? Gitflow Workflow — модель ветвления для git, предполагающая строгую структуру веток: `master` для официальных релизов, `develop` для текущей разработки, `feature/*` для новых функций, `release/*` для подготовки выпуска, `hotfix/*` для экстренных исправлений.

2. В чём суть семантического версионирования? Семантическое версионирование задаёт номер версии в формате `МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ`. Мажорная версия увеличивается при несовместимых изменениях API, минорная — при добавлении новой функциональности без нарушения совместимости, патч — при обратно совместимых исправлениях ошибок.

3. Что такое Conventional Commits? Conventional Commits — спецификация соглашений о написании сообщений коммитов. Каждый коммит имеет тип (`feat`, `fix`, `docs`, `chore` и др.), необязательную область действия и краткое описание. Спецификация тесно связана с семантическим версионированием: тип `feat` соответствует MINOR, `fix` — PATCH, а `BREAKING CHANGE` — MAJOR.

4. Для чего используется инструмент commitizen? Commitizen — интерактивный инструмент для формирования стандартизированных сообщений коммитов. Запускается командой `git cz` и предлагает пользователю выбрать тип коммита, область действия и описание изменений через удобное меню.

5. Для чего используется standard-changelog? Standard-changelog — утилита для автоматической генерации файла `CHANGELOG.md` на основе истории коммитов, написанных по спецификации Conventional Commits. Позволяет автоматически формировать журнал изменений при подготовке каждого релиза.

6 Выводы

В ходе выполнения лабораторной работы были получены навыки работы с расширенными возможностями git: освоена модель ветвления Gitflow, настроены инструменты для создания стандартизированных коммитов (commitizen) и автоматической генерации журнала изменений (standard-changelog). Рабочий репозиторий преобразован в репозиторий с поддержкой git-flow и conventional commits, выполнена публикация релизов на GitHub.

Список литературы